

Detecting heat pumps in aggregate load: slopes, thresholds and resolution

One dataset, built by merging **HEAPO** and the **Swiss** smart-meter dataset. The merge is legitimate because HEAPO's weather station `8jB` is bit-for-bit identical to MeteoSwiss **KLO**, the station the Swiss dataset uses ($\max |\text{diff}| = 0.0000\text{ }^{\circ}\text{C}$ across all 8760 hourly values of 2023), so the two cohorts share a station and their households can sit in the same synthetic substation.

Restrictions carried over from each source:

- * $\geq 90\%$ coverage of the required series over calendar 2023 at 15 min
- * Swiss side: **dwelling only** (Apartment , Single-family house) — no common areas, underground garages, elevators or feed-in systems
- * submetered heat-pump load is the only source of `HP_Peak` ground truth

1702 consumers, 81 with a submetered heat pump.

What this notebook answers

1. Distributions of **slope** and **threshold temperature** from a hockey-stick fit of daily mean load on daily mean temperature.
2. **At what slope can a heat pump be confidently detected**, given that substations with *no* heat pumps also have non-zero slopes.
3. The same fit applied to the **simultaneity factor**, plus the projected **critical temperature** at which SF would reach 1.
4. How fit quality changes with **temporal resolution** (1 h, 4 h, 8 h, 16 h, 24 h), with **spatial aggregation**, and with both together.

Setup

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import hp_pools as hpp
import hp_design as hd
import hp_analysis as ha
from hp_common import hockey_stick, T_BALANCE_BOUNDS

DESIGN_PATH = 'data/design_final.pkl'
FITS_PATH = 'data/final_fits.parquet'

# All False -> load from cache (instant).
REBUILD_POOL = False      # re-read raw data: HEAPO ~7 min, Swiss ~10
min
REGENERATE_DESIGN = False # re-draw the substation design
REFIT = False             # re-fit every substation at every
resolution
```

Substation design

Consumers and heat pumps are varied **independently** on a grid, so an effect can be attributed to one or the other. Households are drawn **without replacement** (a duplicated household would be perfectly correlated with itself and inflate the simultaneity factor), and every member of a substation comes from the **same** weather station.

hp_ratio = 0 cells are generated deliberately: they are the reference for the detection question, since a substation with no heat pumps still has a non-zero temperature slope. Electric heating is left at its **natural prevalence** (~29 % of members have an electric water heater, storage or direct heating) rather than being filtered out — these are meant to look like real substations.

```

In [2]: if REGENERATE_DESIGN or REBUILD_POOL:
        pool = hpp.build_pool_combined(rebuild=REBUILD_POOL)
        design, coverage = hd.generate_design(
            pool,
            n_grid=[10, 20, 50, 100, 200],
            ratio_grid=[0.0, 0.05, 0.10, 0.25, 0.50, 0.75, 1.0],
            n_reps=40, eheat_frac=None, seed=42) # None = natural
        prevalence
        hd.save_design(design, DESIGN_PATH)
        print(f'\n{int(coverage.skipped.sum())} of
        {int(coverage.requested.sum())} '
              f'replicates skipped (N_hp above the 81 submetered heat pumps
        available)')
    else:
        design = hd.load_design(DESIGN_PATH)

    meta = design['meta']
    print(f"{len(meta)} substations | {int((meta.N_hp == 0).sum())} with no
    heat pumps")
    print(f"N_total {sorted(meta.N_total.unique())}")
    print(f"N_hp      {sorted(meta.N_hp.unique())}")
    print(f"distinct households used: {design['n_distinct_households_used']}
    "
          f"of {design['n_distinct_households_available']}")
    print(f"members with other electric heating: "
          f"{100 * meta.N_eheat_realised.sum() / meta.N_total.sum():.1f}%")
    print('\nsubstations per (N_total x N_hp):')
    print(pd.crosstab(meta.N_total, meta.N_hp).to_string())

```

```

1200 substations | 240 with no heat pumps
N_total [10, 20, 50, 100, 200]
N_hp     [0, 1, 2, 5, 8, 10, 12, 15, 20, 25, 38, 50]
distinct households used: 1692 of 1702
members with other electric heating: 29.1%

```

```

substations per (N_total x N_hp):
N_hp      0   1   2   5   8  10  12  15  20  25  38  50
N_total
10         80  40  40  40  40  40   0   0   0   0   0   0
20         40  40  40  40   0  40   0  40  40   0   0   0
50         40   0  40  40   0   0  40   0   0  40  40  40
100        40   0   0  40   0  40   0   0   0  40   0  40
200        40   0   0   0   0  40   0   0  40   0   0  40

```

Fits

For every substation, at each of 1 h / 4 h / 8 h / 16 h / 24 h, a hockey stick $y = \text{base} + \text{slope} \cdot \max(0, T_{\text{threshold}} - T)$ is fitted for two responses:

* **Load** — mean net load against mean temperature (what a DSO observes)

* **SF** — simultaneity factor $\text{HP_Load} / \text{HP_Peak}$ against mean temperature (submetered ground truth; undefined where there are no heat pumps)

`slope_per_peak` and `slope_per_base` normalise the slope by an **observable** quantity. This matters for detection: the slope grows with ordinary consumers as well as with heat pumps, and a DSO cannot divide by a household count it does not have.

```
In [3]: if REFIT or REGENERATE_DESIGN:
        fits = ha.fit_all(design)
        fits.to_parquet(FITS_PATH)
    else:
        fits = pd.read_parquet(FITS_PATH)

    load_fits = fits[fits.response == 'Load']
    sf_fits = fits[fits.response == 'SF']
    print(f'{len(fits)} fits, {int(fits.failed.sum())} failed')
    print(f'  Load: {len(load_fits)}   SF: {len(sf_fits)}   '
          f'reolutions: {ha.LABELS}')

10800 fits, 0 failed
  Load: 6000   SF: 4800   resolutions: ['1 h', '4 h', '8 h', '16
h', '24 h']
```

1. Net-load hockey stick — slope and threshold temperature

Daily means (24 h). The slope distribution is shown separately for substations with and without heat pumps, because their overlap is exactly what limits detection.

```

In [4]: print(ha.describe_params(fits, 'Load').round(3).to_string(index=False))

d24 = load_fits[(load_fits.resolution == '24 h') & (~load_fits.failed)]
no_hp = d24[d24.N_hp == 0]
with_hp = d24[d24.N_hp > 0]

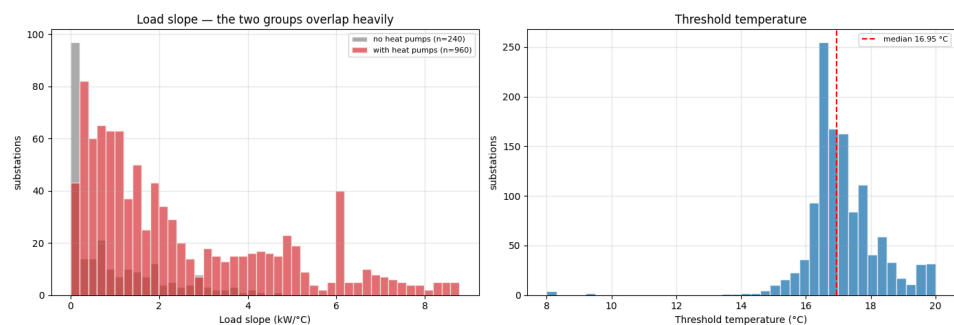
fig, axs = plt.subplots(1, 2, figsize=(14, 4.8))
bins = np.linspace(0, d24.slope.quantile(.99), 45)
axs[0].hist(no_hp.slope, bins=bins, alpha=.65, label=f'no heat pumps (n={len(no_hp)})',
            color='tab:grey', edgecolor='white')
axs[0].hist(with_hp.slope, bins=bins, alpha=.65, label=f'with heat pumps (n={len(with_hp)})',
            color='tab:red', edgecolor='white')
axs[0].set_xlabel('Load slope (kW/°C)');
axs[0].set_ylabel('substations')
axs[0].set_title('Load slope – the two groups overlap heavily')
axs[0].grid(alpha=.3); axs[0].legend(fontsize=8)

axs[1].hist(d24.T_threshold, bins=40, color='tab:blue', alpha=.75,
            edgecolor='white')
axs[1].axvline(d24.T_threshold.median(), color='red', ls='--',
               label=f'median {d24.T_threshold.median():.2f} °C')
axs[1].set_xlabel('Threshold temperature (°C)');
axs[1].set_ylabel('substations')
axs[1].set_title('Threshold temperature')
axs[1].grid(alpha=.3); axs[1].legend(fontsize=8)
fig.tight_layout(); plt.show()

print(f'median slope: no HP {no_hp.slope.median():.3f} vs with HP '
      f'{with_hp.slope.median():.3f} kW/°C')
print(f'threshold temperature is stable at '
      f'{d24.T_threshold.median():.2f} °C '
      f'f'(IQR {d24.T_threshold.quantile(.25):.2f}-'
      f'{d24.T_threshold.quantile(.75):.2f}))')

```

parameter	n	median	q25	q75	min	max
slope	1200	1.419	0.579	3.424	0.000	9.839
T_threshold	1200	16.946	16.547	17.662	8.000	20.000
T_crit	960	-29.429	-39.471	-10.135	-416.987	14.953



median slope: no HP 0.536 vs with HP 1.733 kW/°C
 threshold temperature is stable at 16.95 °C (IQR 16.55-17.66)

2. At what slope is a heat pump detectable?

A raw slope threshold cannot work, because the slope grows with **both** heat pumps (~0.10–0.15 kW/°C each) and ordinary consumers (~0.006 kW/°C each). A large substation with no heat pumps out-slopes a small one with several — visible in the table below, where the no-heat-pump column reaches 2.89 kW/°C at 200 consumers.

Detection therefore needs the slope normalised by something observable without a household census: peak load or base load, never the consumer count. Each candidate is scored by AUC, and the operating threshold is set at 95 % specificity — i.e. at most 5 % of heat-pump-free substations are flagged.

```
In [5]: print('Median Load slope (kW/°C) at 24 h – rows = consumers, columns =
heat pumps:')
print(d24.pivot_table(index='N_total', columns='N_hp', values='slope',
aggfunc='median').round(2).to_string())
print()
print('Candidate detection statistics (24 h, specificity 0.95):')
print(ha.statistic_comparison(load_fits).round(4).to_string(index=False)
)
```

Median Load slope (kW/°C) at 24 h – rows = consumers, columns =
heat pumps:

N_hp	0	1	2	5	8	10	12	15	20
25	38	50							
N_total									
10	0.08	0.19	0.26	0.57	0.85	1.05	NaN	NaN	NaN
NaN	NaN	NaN							
20	0.13	0.22	0.34	0.66	NaN	1.12	NaN	1.55	2.00
NaN	NaN	NaN							
50	0.71	NaN	0.91	1.31	NaN	NaN	1.99	NaN	NaN
3.38	4.96	6.11							
100	1.51	NaN	NaN	2.04	NaN	2.44	NaN	NaN	NaN
4.21	NaN	6.84							
200	2.89	NaN	NaN	NaN	NaN	3.94	NaN	NaN	5.19
NaN	NaN	8.40							

Candidate detection statistics (24 h, specificity 0.95):

statistic	auc	threshold	sensitivity	observed_fpr
slope	0.7579	3.1701	0.3219	0.05
slope_per_peak	0.8453	0.0287	0.4083	0.05
slope_per_base	0.8472	0.0891	0.3906	0.05

```

In [6]: det_raw = ha.detection_threshold(load_fits, 'slope')
det_norm = ha.detection_threshold(load_fits, 'slope_per_peak')

for name, det, unit in [('raw slope', det_raw, ' kW/°C'),
                        ('slope / peak load', det_norm, ' 1/°C')]:
    print(f'{name}: detect when statistic > {det["threshold"]:.4f}
{unit}')
        f' -> sensitivity {det["sensitivity"]:.3f} at FPR
{det["observed_fpr"]:.3f}')
    print('    by penetration: ' +
          ' '.join(f'{k:.0%}={v:.2f}' for k, v in
det['by_hp_ratio'].items()))
    print()

fig, axs = plt.subplots(1, 2, figsize=(14, 4.8))
for stat, lab in [('slope', 'raw slope'), ('slope_per_peak', 'slope /
peak load'),
                  ('slope_per_base', 'slope / base load')]:
    curve, auc = ha.roc(load_fits, stat)
    axs[0].plot(curve.fpr, curve.tpr, label=f'{lab} (AUC {auc:.3f})')
    axs[0].plot([0, 1], [0, 1], 'k:', lw=.8)
    axs[0].set_xlabel('false-positive rate (heat-pump-free flagged)')
    axs[0].set_ylabel('true-positive rate')
    axs[0].set_title('Detecting the presence of heat pumps (24 h)')
    axs[0].grid(alpha=.3); axs[0].legend(fontsize=8)

for det, lab in [(det_raw, 'raw slope'), (det_norm, 'slope / peak
load')]:
    s = det['by_hp_ratio']
    axs[1].plot(s.index * 100, s.values, 'o-', label=lab)
    axs[1].axhline(.8, color='grey', ls='-.', lw=.8, label='80 % detection')
    axs[1].set_xlabel('heat-pump penetration (%)')
    axs[1].set_ylabel('detection rate at 95 % specificity')
    axs[1].set_title('Detection vs penetration')
    axs[1].set_ylim(0, 1.02); axs[1].grid(alpha=.3);
    axs[1].legend(fontsize=8)
fig.tight_layout(); plt.show()

```

```

raw slope: detect when statistic > 3.1701 kW/°C -> sensitivity
0.322 at FPR 0.050

```

```

    by penetration: 5%=0.22 10%=0.22 25%=0.40 50%=0.44 75%=0.33
100%=0.33

```

```

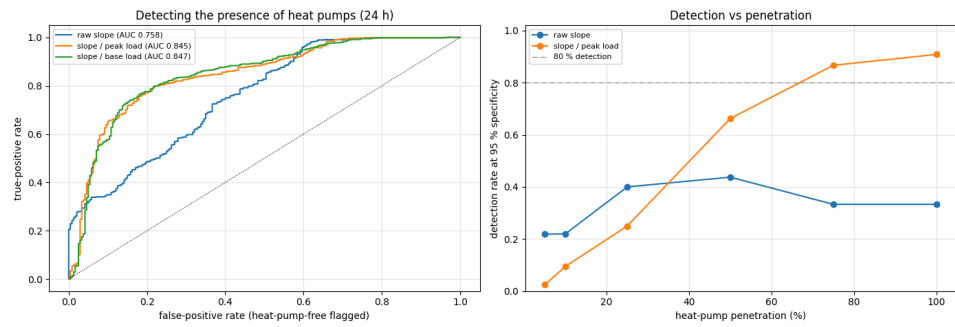
slope / peak load: detect when statistic > 0.0287 1/°C ->
sensitivity 0.408 at FPR 0.050

```

```

    by penetration: 5%=0.03 10%=0.10 25%=0.25 50%=0.66 75%=0.87
100%=0.91

```



```
In [7]: # Does averaging help or hurt detection? R2 improves with coarser data,
# so the
# expectation would be yes -- the answer is the opposite.
rows = []
for r in ha.LABELS:
    _, auc = ha.roc(load_fits, 'slope_per_peak', r)
    det = ha.detection_threshold(load_fits, 'slope_per_peak', r)
    rows.append({'resolution': r, 'auc': auc, 'threshold':
det['threshold'],
                'sensitivity': det['sensitivity']})
det_by_res = pd.DataFrame(rows)
print(det_by_res.round(4).to_string(index=False))
print()
print('Detectability PEAKS at 1-4 h and falls towards daily, even though
the fit')
print('itself keeps improving. Averaging sharpens both groups: the heat-
pump-free')
print('slopes tighten and rise too, so the discriminating gap narrows.
Good fit')
print('quality and good detectability are not the same objective.')
```

resolution	auc	threshold	sensitivity
1 h	0.8640	0.0174	0.6260
4 h	0.8718	0.0203	0.6448
8 h	0.8567	0.0247	0.5677
16 h	0.8588	0.0255	0.5469
24 h	0.8453	0.0287	0.4083

Detectability PEAKS at 1-4 h and falls towards daily, even though
the fit
itself keeps improving. Averaging sharpens both groups: the heat-
pump-free
slopes tighten and rise too, so the discriminating gap narrows.
Good fit
quality and good detectability are not the same objective.

3. Simultaneity factor — slope, threshold and critical temperature

$SF = HP_Load / HP_Peak$, bounded in $[0, 1]$ because households are drawn without replacement. Defined only where heat pumps exist.

The **critical temperature** is the extrapolated point at which the heat pumps would together reach full installed capacity:

* from the SF fit: $T_crit = T_threshold - (1 - base) / slope$

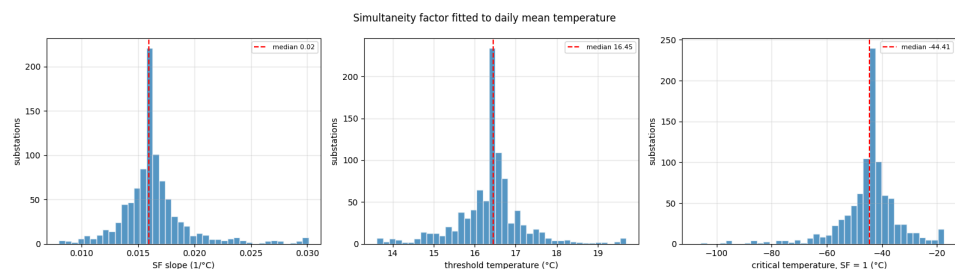
* from the load fit: $T_crit = T_threshold - HP_Peak / slope$

Comparing the two says how far the observable route can be trusted.

```
In [8]: print(ha.describe_params(fits, 'SF').round(3).to_string(index=False))

s24 = sf_fits[(sf_fits.resolution == '24 h') & (~sf_fits.failed)]
specs = [('slope', 'SF slope (1/°C)'), ('T_threshold', 'threshold
temperature (°C)'),
         ('T_crit', 'critical temperature, SF = 1 (°C)')]
fig, axs = plt.subplots(1, 3, figsize=(16, 4.5))
for ax, (col, lab) in zip(axs, specs):
    v = s24[col].replace([np.inf, -np.inf], np.nan).dropna()
    lo, hi = v.quantile(.01), v.quantile(.99)
    ax.hist(v, bins=40, range=(lo, hi), color='tab:blue', alpha=.75,
            edgecolor='white')
    ax.axvline(v.median(), color='red', ls='--', label=f'median
{v.median():.2f}')
    ax.set_xlabel(lab); ax.set_ylabel('substations'); ax.grid(alpha=.3)
    ax.legend(fontsize=8)
fig.suptitle('Simultaneity factor fitted to daily mean temperature')
fig.tight_layout(); plt.show()
```

parameter	n	median	q25	q75	min	max
slope	960	0.016	0.015	0.017	0.006	0.037
T_threshold	960	16.450	16.052	16.694	11.522	20.000
T_crit	960	-44.408	-47.925	-40.092	-135.254	-10.489



```
In [9]: # How far is SF = 1 from anything observed?
sf24 = s24.assign(SF_at_coldest=s24.base + s24.slope *
                  np.maximum(0.0, s24.T_threshold - s24.T_min_obs))
print(f'coldest observed daily mean : {s24.T_min_obs.median():.1f} °C')
print(f'modelled SF on that day      : {sf24.SF_at_coldest.median():.2f}'
      f' [IQR {sf24.SF_at_coldest.quantile(.25):.2f}, '
      f'{sf24.SF_at_coldest.quantile(.75):.2f}]]')
print(f'substations reaching SF = 1 : {int((sf24.SF_at_coldest >=
1).sum())} '
      f'of {len(sf24)}')
print()
ld = load_fits[(load_fits.resolution == '24 h') & (load_fits.N_hp > 0)]
cmp = pd.DataFrame({
    'SF_T_crit': s24.groupby('hp_ratio').T_crit.median(),
    'Load_T_crit': ld.groupby('hp_ratio').T_crit.median()})
cmp['gap'] = cmp.Load_T_crit - cmp.SF_T_crit
print('critical temperature: ground truth (SF) vs observable (net
load)')
print(cmp.round(1).to_string())
print()
print('SF = 1 is never approached -- the projection sits far below any
Swiss')
print('climate, so the empirically supported design point is SF ~ 0.4-
0.5.')
print('The observable route is most optimistic where heat pumps are
sparse and')
print('converges on ground truth only near full penetration.')
```

```
coldest observed daily mean : -8.2 °C
modelled SF on that day      : 0.42 [IQR 0.39, 0.45]
substations reaching SF = 1 : 0 of 960
```

```
critical temperature: ground truth (SF) vs observable (net load)
      SF_T_crit  Load_T_crit  gap
hp_ratio
0.05         -41.4          0.5  41.9
0.10         -42.6         -11.7  30.9
0.25         -44.4         -27.0  17.4
0.50         -44.4         -36.1   8.3
0.75         -44.4         -40.0   4.4
1.00         -44.4         -41.3   3.1
```

```
SF = 1 is never approached -- the projection sits far below any
Swiss
climate, so the empirically supported design point is SF ~ 0.4-
0.5.
The observable route is most optimistic where heat pumps are
sparse and
converges on ground truth only near full penetration.
```

4. Fit quality vs resolution and aggregation

Three questions at once: does coarser averaging help, does a bigger substation help, and do they act on the same thing? The mixed model separates the consumer count from the heat-pump count, which a plain size variable cannot do.

```
In [10]: for resp in ['Load', 'SF']:
          print(f'median R2 – {resp} (rows = consumers, columns =
          resolution)')
          print(ha.quality_grid(fits, resp).round(3).to_string())
          print()
```

```
median R2 – Load (rows = consumers, columns = resolution)
resolution    1 h    4 h    8 h   16 h   24 h
N_total
10           0.310  0.476  0.580  0.739  0.811
20           0.444  0.610  0.687  0.818  0.891
50           0.607  0.726  0.789  0.890  0.934
100          0.561  0.668  0.760  0.874  0.927
200          0.559  0.669  0.751  0.881  0.932
```

```
median R2 – SF (rows = consumers, columns = resolution)
resolution    1 h    4 h    8 h   16 h   24 h
N_total
10           0.681  0.844  0.906  0.937  0.952
20           0.745  0.865  0.919  0.950  0.964
50           0.842  0.921  0.944  0.963  0.972
100          0.832  0.920  0.945  0.962  0.972
200          0.849  0.921  0.948  0.966  0.974
```

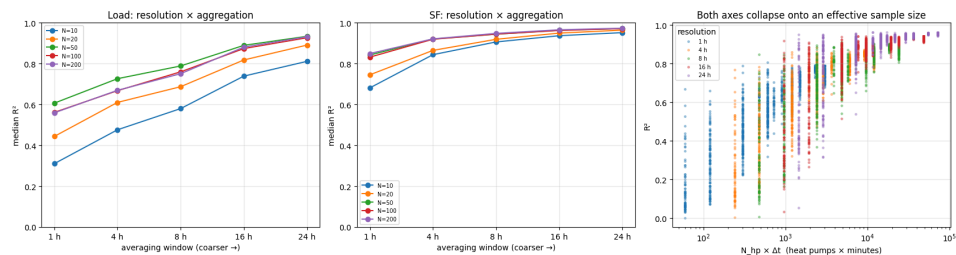
```

In [11]: fig, axs = plt.subplots(1, 3, figsize=(17, 4.6))

for resp, ax in zip(['Load', 'SF'], axs[:2]):
    grid = ha.quality_grid(fits, resp)
    for n in grid.index:
        ax.plot(range(len(grid.columns)), grid.loc[n], 'o-', label=f'N={n}')
    ax.set_xticks(range(len(grid.columns)))
    ax.set_xticklabels(grid.columns)
    ax.set_xlabel('averaging window (coarser →)')
    ax.set_ylabel('median R2')
    ax.set_title(f'{resp}: resolution × aggregation')
    ax.set_ylim(0, 1); ax.grid(alpha=.3); ax.legend(fontsize=7)

# combined axis: effective sample size N_hp * dt
ok = fits[(~fits.failed) & (fits.response == 'Load') & (fits.N_hp > 0)].copy()
ok['neff'] = ok.N_hp * ok.dt_minutes
for r in ha.LABELS:
    s = ok[ok.resolution == r]
    axs[2].scatter(s.neff, s.r2, s=6, alpha=.35, label=r)
axs[2].set_xscale('log')
axs[2].set_xlabel('N_hp × Δt (heat pumps × minutes)')
axs[2].set_ylabel('R2')
axs[2].set_title('Both axes collapse onto an effective sample size')
axs[2].grid(alpha=.3); axs[2].legend(fontsize=7, title='resolution')
fig.tight_layout(); plt.show()

```



```
In [12]: print('logit(R2) ~ log Δt + log N_total + log N_hp (substation random
effect)')
for resp in ['Load', 'SF']:
    print(f'\n--- {resp} ---')
    print(ha.quality_model(fits, resp).round(3).to_string(index=False))
print()
print('log_dt > 0 : coarser averaging always helps')
print('log_Nhp > 0 : more heat pumps help, and this term dominates')
print('log_N      : for net load slightly NEGATIVE -- extra non-HP
consumers add')
print('              load uncorrelated with temperature and dilute the
signal.')
print('              For SF it is ~0, as it must be: SF is already
normalised by')
print('              installed capacity, so substation size cannot
matter.')
```

logit(R²) ~ log Δt + log N_total + log N_hp (substation random effect)

--- Load ---

term	coef	lo	hi
log_dt	0.656	0.649	0.662
log_N	-0.111	-0.150	-0.072
log_Nhp	0.753	0.719	0.786

--- SF ---

term	coef	lo	hi
log_dt	0.657	0.651	0.664
log_N	0.017	-0.012	0.046
log_Nhp	0.467	0.442	0.492

log_dt > 0 : coarser averaging always helps
log_Nhp > 0 : more heat pumps help, and this term dominates
log_N : for net load slightly NEGATIVE -- extra non-HP
consumers add
 load uncorrelated with temperature and dilute the
signal.
 For SF it is ~0, as it must be: SF is already
normalised by
 installed capacity, so substation size cannot
matter.

5. Out-of-sample validation on WPUQ (Germany)

Everything above is Swiss. WPUQ is an independent test: 37 German single-family houses, each with separately metered `HEATPUMP` and `HOUSEHOLD` circuits (verified independent — $r = -0.04$, the heat-pump series is strongly seasonal and the household series is not). Different climate, buildings, and measurement chain.

Substations are built the same way and fitted with the same code. The real test is **transfer**: the detection threshold calibrated on the Swiss data is applied to WPUQ **unchanged** — no refitting, no recalibration.

Zero-heat-pump substations are formed from the `HOUSEHOLD` circuits alone. Note this makes them *synthetic* HP-free references, as HEAPO's are: every WPUQ house actually owns a heat pump. That matters for interpreting the false-positive rate.

```
In [13]: WPUQ_DESIGN = 'data/design_wpuq_val.pkl'
WPUQ_FITS = 'data/wpuq_val_fits.parquet'
REGENERATE_WPUQ = False

if REGENERATE_WPUQ:
    wpuq_pool = hpp.build_pool_wpuq(rebuild=REBUILD_POOL)
    wpuq_design, wpuq_cov = hd.generate_design(
        wpuq_pool, n_grid=[10, 20, 37],
        ratio_grid=[0.0, 0.05, 0.10, 0.25, 0.50, 0.75, 1.0],
        n_reps=40, eheat_frac=None, seed=42, verbose=False)
    hd.save_design(wpuq_design, WPUQ_DESIGN)
    wpuq_fits = ha.fit_all(wpuq_design, verbose=False)
    wpuq_fits.to_parquet(WPUQ_FITS)
else:
    wpuq_design = hd.load_design(WPUQ_DESIGN)
    wpuq_fits = pd.read_parquet(WPUQ_FITS)

wmeta = wpuq_design['meta']
wpuq_load = wpuq_fits[wpuq_fits.response == 'Load']
wpuq_sf = wpuq_fits[wpuq_fits.response == 'SF']
print(f'{len(wmeta)} WPUQ substations | '
      f'{int((wmeta.N_hp == 0).sum())} with no heat pumps | '
      f'{wpuq_design["n_distinct_households_used"]} households')
print(f'N_total {sorted(wmeta.N_total.unique())} | N_hp '
      f'{sorted(wmeta.N_hp.unique())}')
```

```
840 WPUQ substations | 160 with no heat pumps | 37 households
N_total [10, 20, 37] | N_hp [0, 1, 2, 4, 5, 8, 9, 10, 15, 18, 20,
28, 37]
```

```
In [14]: # THE TRANSFER TEST: Swiss thresholds, applied to WPUQ without
recalibration.
rows = []
for res in ha.LABELS:
    det = ha.detection_threshold(load_fits, 'slope_per_peak', res)
    thr = det['threshold']
    w = wpuq_load[(wpuq_load.resolution == res) & (~wpuq_load.failed)]
    w = w[np.isfinite(w.slope_per_peak)]
    pos, neg = w[w.N_hp > 0], w[w.N_hp == 0]
    rows.append({'resolution': res, 'swiss_threshold': thr,
                'swiss_sens': det['sensitivity'], 'swiss_fpr':
det['observed_fpr'],
                'wpuq_sens': (pos.slope_per_peak >= thr).mean(),
                'wpuq_fpr': (neg.slope_per_peak >= thr).mean()})
transfer = pd.DataFrame(rows)
print(transfer.round(4).to_string(index=False))
print()
print('The Swiss threshold generalises: it flags roughly half of the
WPUQ')
print('heat-pump substations with ZERO false positives at every
resolution.')
print('It transfers CONSERVATIVELY -- the realised false-positive rate
is 0.00')
print('rather than the 0.05 it was tuned for, which costs some
sensitivity at')
print('low penetration but never mislabels a heat-pump-free
substation.')
```

resolution	swiss_threshold	swiss_sens	swiss_fpr	wpuq_sens
wpuq_fpr				
1 h	0.0174	0.6260	0.05	0.5632
0.0				
4 h	0.0203	0.6448	0.05	0.5647
0.0				
8 h	0.0247	0.5677	0.05	0.5191
0.0				
16 h	0.0255	0.5469	0.05	0.6162
0.0				
24 h	0.0287	0.4083	0.05	0.5647
0.0				

The Swiss threshold generalises: it flags roughly half of the WPUQ heat-pump substations with ZERO false positives at every resolution.

It transfers CONSERVATIVELY -- the realised false-positive rate is 0.00

rather than the 0.05 it was tuned for, which costs some sensitivity at

low penetration but never mislabels a heat-pump-free substation.

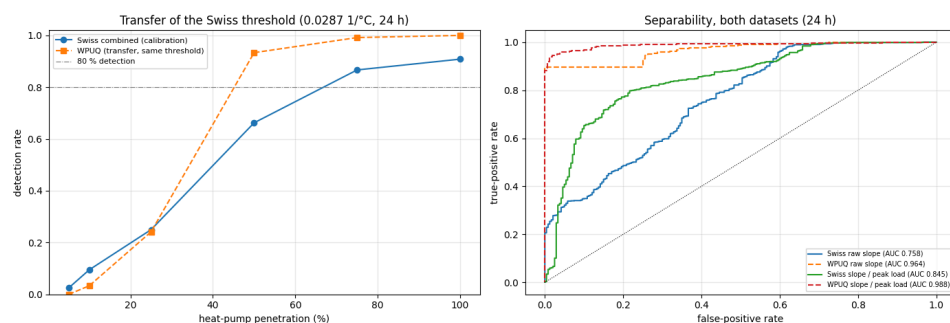
```

In [15]: # Detection vs penetration, both datasets, at the Swiss 24 h threshold.
det24 = ha.detection_threshold(load_fits, 'slope_per_peak', '24 h')
thr24 = det24['threshold']
w24 = wpuq_load[(wpuq_load.resolution == '24 h') & (~wpuq_load.failed) &
                 (wpuq_load.N_hp > 0)]
wpuq_by_ratio = (w24.assign(flag=w24.slope_per_peak >= thr24)
                 .groupby('hp_ratio').flag.mean())

fig, axs = plt.subplots(1, 2, figsize=(14, 4.8))
axs[0].plot(det24['by_hp_ratio'].index * 100,
            det24['by_hp_ratio'].values,
            'o-', label='Swiss combined (calibration)')
axs[0].plot(wpuq_by_ratio.index * 100, wpuq_by_ratio.values, 's--',
            label='WPUQ (transfer, same threshold)')
axs[0].axhline(.8, color='grey', ls='-.', lw=.8, label='80 % detection')
axs[0].set_xlabel('heat-pump penetration (%)')
axs[0].set_ylabel('detection rate')
axs[0].set_title(f'Transfer of the Swiss threshold ({thr24:.4f} 1/°C, 24 h)')
axs[0].set_ylim(0, 1.02); axs[0].grid(alpha=.3);
axs[0].legend(fontsize=8)

for stat, lab in [('slope', 'raw slope'), ('slope_per_peak', 'slope /
peak load')]:
    c_s, a_s = ha.roc(load_fits, stat)
    c_w, a_w = ha.roc(wpuq_load, stat)
    axs[1].plot(c_s.fpr, c_s.tpr, '-', label=f'Swiss {lab} (AUC
{a_s:.3f})')
    axs[1].plot(c_w.fpr, c_w.tpr, '--', label=f'WPUQ {lab} (AUC
{a_w:.3f})')
axs[1].plot([0, 1], [0, 1], 'k:', lw=.8)
axs[1].set_xlabel('false-positive rate'); axs[1].set_ylabel('true-
positive rate')
axs[1].set_title('Separability, both datasets (24 h)')
axs[1].grid(alpha=.3); axs[1].legend(fontsize=7)
fig.tight_layout(); plt.show()

```



```

In [16]: # Do the fitted parameters agree?
print(f'{"parameter (24 h)":26s} {"Swiss combined":>26s} {"WPUQ":>26s}')
for resp, cols in [('Load', ['slope', 'T_threshold']),
                    ('SF', ['slope', 'T_threshold', 'T_crit'])]:
    for c in cols:
        cellvals = []
        for df in (fits, wpuq_fits):
            d = df[(df.response == resp) & (df.resolution == '24 h') &
                    (~df.failed)]
            v = d[c].replace([np.inf, -np.inf], np.nan).dropna()
            cellvals.append(f'{v.median():.3f} [{v.quantile(.25):.3f},
{v.quantile(.75):.3f}]')
            print(f'{resp + " " + c:26s} {cellvals[0]:>26s}
{cellvals[1]:>26s}')

import statsmodels.formula.api as smf
print('\nslope ~ N_hp + N_total    (24 h net load)')
for lab, df in [('Swiss', fits), ('WPUQ', wpuq_fits)]:
    d = df[(df.response == 'Load') & (df.resolution == '24 h') &
            (~df.failed)]
    m = smf.ols('slope ~ N_hp + N_total', data=d).fit()
    print(f' {lab:6s} per heat pump {m.params["N_hp"]:+.4f} kW/°C    '
          f'per consumer {m.params["N_total"]:+.5f}    R²
{m.rsquared:.3f}')

print('\nlogit(R²) ~ log Δt + log N_total + log N_hp    (net load)')
for lab, df in [('Swiss', fits), ('WPUQ', wpuq_fits)]:
    q = ha.quality_model(df, 'Load').set_index('term')
    print(f' {lab:6s} Δt {q.loc["log_dt", "coef"]:+.3f}    '
          f'N_total {q.loc["log_N", "coef"]:+.3f}    '
          f'N_hp {q.loc["log_Nhp", "coef"]:+.3f}')

```

parameter (24 h)	Swiss combined	
WPUQ		
Load slope	1.419 [0.579, 3.424]	0.533
[0.179, 1.293]		
Load T_threshold	16.946 [16.547, 17.662]	15.906
[13.945, 19.032]		
SF slope	0.016 [0.015, 0.017]	0.013
[0.012, 0.015]		
SF T_threshold	16.450 [16.052, 16.694]	13.267
[12.651, 14.030]		
SF T_crit	-44.408 [-47.925, -40.092]	-60.426
[-70.731, -52.023]		

slope ~ N_hp + N_total (24 h net load)			
Swiss	per heat pump +0.1082 kW/°C	per consumer +0.01543	R²
0.970			
WPUQ	per heat pump +0.1011 kW/°C	per consumer +0.00308	R²
0.986			

logit(R²) ~ log Δt + log N_total + log N_hp (net load)			
Swiss	Δt +0.656	N_total -0.111	N_hp +0.753
WPUQ	Δt +0.573	N_total -0.609	N_hp +0.960

What WPUQ confirms, and what it does not

Confirmed — the detection statistic transfers. The Swiss threshold on slope / peak load, applied to German data without recalibration, flags heat pumps with **zero false positives** at every resolution and reaches 0.93 / 0.99 / 1.00 detection at 50 / 75 / 100 % penetration. Normalising by an observable is what makes this portable; a raw-slope threshold is tied to substation size and would not survive the move.

Confirmed — the physical scale of a heat pump. The slope decomposition gives **+0.101 kW/°C per heat pump** in WPUQ against **+0.108** in the Swiss data — a 7 % difference across two countries. That is the most transferable number here.

Confirmed — the scaling laws. Coarser averaging helps (Δt +0.57 vs +0.66), more heat pumps help (+0.96 vs +0.75), and additional non-HP consumers dilute (−0.61 vs −0.11) in both datasets.

Not transferable — the threshold temperature. 13.3 °C (WPUQ) against 16.5 °C (Swiss) for the SF fit, and the SF = 1 projection moves from −44 °C to −60 °C. Balance temperature is a property of the building stock and climate, so it has to be estimated locally rather than carried over.

The caveat that matters. WPUQ separates far more easily than the Swiss data (AUC 0.99 vs 0.85), and its false-positive rate is 0.00 rather than 0.05. This is not because German heat pumps are easier to see — it is because every WPUQ house owns a heat pump, so its zero-heat-pump substations are *constructed* by omitting the heat-pump circuit. They contain no electric water heating, no resistive backup, no gas-boiler circulation pumps. The Swiss combined dataset uses **real** heat-pump-free dwellings, which do have those loads — 29 % of members carry some electric heating.

So the Swiss AUC of 0.85 is the realistic figure and WPUQ's 0.99 is optimistic. WPUQ validates that the method transfers across climate and metering; it cannot validate the false-positive rate, because it has no genuine negatives.

Summary

```
In [17]: print('=' * 78)
print('COMBINED HEAPO + SWISS - '
      f'{len(meta)} substations, {design["n_distinct_households_used"]}'
      'households')
print('=' * 78)

print('\nNET LOAD, daily means')
print(f' threshold temperature : {d24.T_threshold.median():.2f} °C '
      f'[IQR {d24.T_threshold.quantile(.25):.2f},
{d24.T_threshold.quantile(.75):.2f}]]')
print(f' slope, no heat pumps : {no_hp.slope.median():.3f} kW/°C '
      f'[IQR {no_hp.slope.quantile(.25):.3f},
{no_hp.slope.quantile(.75):.3f}]]')
print(f' slope, with heat pumps: {with_hp.slope.median():.3f} kW/°C '
      f'[IQR {with_hp.slope.quantile(.25):.3f},
{with_hp.slope.quantile(.75):.3f}]]')

print('\nDETECTION (95 % specificity)')
print(f' raw slope : > {det_raw["threshold"]:.3f} kW/°C, '
      f'sensitivity {det_raw["sensitivity"]:.2f} (AUC '
      f'{ha.roc(load_fits, "slope")[1]:.3f}'))
print(f' slope / peak load : > {det_norm["threshold"]:.4f} 1/°C, '
      f'sensitivity {det_norm["sensitivity"]:.2f} (AUC '
      f'{ha.roc(load_fits, "slope_per_peak")[1]:.3f}'))
_b = det_norm['by_hp_ratio']
_ok = _b[_b > 0.8]
print(f' penetration needed for 80 % detection: '
      f'{"{(_ok.index.min():.0%)" if len(_ok) else "not reached"}')')
print(f' best resolution for detection: '
      f'{det_by_res.loc[det_by_res.auc.idxmax(), "resolution"]} '
      f'(AUC {det_by_res.auc.max():.3f}) vs 24 h '
      f'(AUC {det_by_res.set_index("resolution").loc["24 h",
"auc"]:.3f}'))

print('\nSIMULTANEITY FACTOR, daily means')
print(f' threshold temperature : {s24.T_threshold.median():.2f} °C '
      f'[IQR {s24.T_threshold.quantile(.25):.2f},
{s24.T_threshold.quantile(.75):.2f}]]')
print(f' slope : {s24.slope.median():.4f} 1/°C '
      f'[IQR {s24.slope.quantile(.25):.4f},
{s24.slope.quantile(.75):.4f}]]')
print(f' T at SF = 1 : {s24.T_crit.median():.1f} °C '
      f'[IQR {s24.T_crit.quantile(.25):.1f},
{s24.T_crit.quantile(.75):.1f}] (extrapolated)')
print(f' SF at coldest observed: {sf24.SF_at_coldest.median():.2f} '
      f'-> full coincidence never observed')

print('\nFIT QUALITY')
_qm = ha.quality_model(fits, 'Load').set_index('term')
print(f' per e-fold of Δt : {_qm.loc["log_dt", "coef"]:.3f} logit
R²')
```

```

print(f' per e-fold of N_hp : {_qm.loc["log_Nhp", "coef"]:+.3f}')
print(f' per e-fold of N      : {_qm.loc["log_N", "coef"]:+.3f} (non-HP
consumers dilute)')
print('\nSubstations are pseudo-replicates from a limited household
pool; the')
print('effective sample size is well below the substation count.')

```

```

=====
=====
COMBINED HEAPO + SWISS - 1200 substations, 1692 households
=====
=====

```

NET LOAD, daily means

```

threshold temperature : 16.95 °C [IQR 16.55, 17.66]
slope, no heat pumps  : 0.536 kW/°C [IQR 0.103, 1.546]
slope, with heat pumps: 1.733 kW/°C [IQR 0.765, 4.105]

```

DETECTION (95 % specificity)

```

raw slope          : > 3.170 kW/°C, sensitivity 0.32 (AUC
0.758)
slope / peak load  : > 0.0287 1/°C, sensitivity 0.41 (AUC
0.845)
penetration needed for 80 % detection: 75%
best resolution for detection: 4 h (AUC 0.872) vs 24 h (AUC
0.845)

```

SIMULTANEITY FACTOR, daily means

```

threshold temperature : 16.45 °C [IQR 16.05, 16.69]
slope                 : 0.0160 1/°C [IQR 0.0150, 0.0172]
T at SF = 1          : -44.4 °C [IQR -47.9, -40.1]
(extrapolated)
SF at coldest observed: 0.42 -> full coincidence never observed

```

FIT QUALITY

```

per e-fold of Δt    : +0.656 logit R²
per e-fold of N_hp  : +0.753
per e-fold of N      : -0.111 (non-HP consumers dilute)

```

Substations are pseudo-replicates from a limited household pool;
the
effective sample size is well below the substation count.